252-0027

Einführung in die Programmierung

Übungen

Woche 6: References, Recursion, Precondition

Jhalak Yadav
Departement Informatik
ETH Zürich

Organisatorisches

- Mein Name: Jhalak Yadav
- Bei Fragen: jyadav@student.ethz.ch
  - Mails bitte mit «[EProg26]» im Betreff
- Neue Aufgaben: **Dienstag Abend** (im Normalfall)
- Abgabe der Übungen bis **Dienstag Abend (23:59)** Folgewoche
  - Abgabe immer via Git
  - Lösungen in separatem Projekt auf Git

jhalakyadav.github.io

Referenzen

Value vs Referenzen

- Variablen enthalten entweder einen **Wert** (Value) oder eine **Referenz** (Reference).
- Variablen enthalten **nie** Objekte.
- Referenzen zeigen auf das Objekt im Speicher.

Value vs Reference - Beispiel

```
1  public class Example {
2      public static void setX(int x) {
3          x = 2;
4      }
5
6      public static void main(String[] args) {
7          int x = 3;
8          setX(x);
9          System.out.println(x); ⟶ 3
10     }
11 }
```

Value vs Reference - Beispiel

```
1  public class Example {
2      public static void setX(int x) {
3          x = 2;
4      }
5
6      public static void main(String[] args) {
7          int x = 3;
8          setX(x);
9          System.out.println(x);
10     }
11 }
```

Scope setX

Scope main

Value vs Reference

- Referenzen befolgen auch Value Semantics:
  - In Java werden Variablen, die auf Objekte verweisen, tatsächlich als Referenzen behandelt.
  - Beim Zuweisen einer Referenz zu einer anderen wird nur die Speicheradresse (Referenz) kopiert, nicht das tatsächliche Objekt.
  - Jede Referenz speichert die Adresse des Objekts, auf das sie zeigt.
- Nur die Objekte selbst ermöglichen Reference Semantics:
  - Mehrere Referenzen können auf dasselbe Objekt im Speicher zeigen (z.B. A obj1 = new A(); A obj2 = obj1;).
  - Referenzen in Java sind nicht selbstständige Kopien des Objekts, sondern Zeiger auf den gleichen Speicherort.

Value vs Reference - Beispiel

```
1  import java.util.Arrays;
2
3  public class Example {
4      public static void setX(int[] x) {
5          x = new int[] {4, 5, 6};
6      }
7
8      public static void main(String[] args) {
9          int[] x = new int[] {1, 2, 3};
10         setX(x);
11         String xStr = Arrays.toString(x);
12         System.out.println(xStr);
13     }
14 }
```

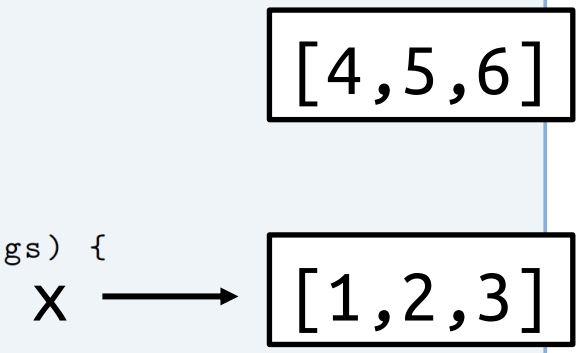
x

[4,5,6]

[1,2,3]

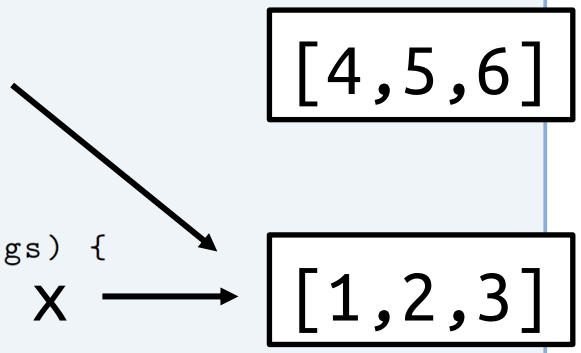
Value vs Reference - Beispiel

```
1  import java.util.Arrays;
2
3  public class Example {
4      public static void setX(int[] x) {
5          x = new int[] {4, 5, 6};
6      }
7
8      public static void main(String[] args) {
9          int[] x = new int[] {1, 2, 3};
10         setX(x);
11         String xStr = Arrays.toString(x);
12         System.out.println(xStr);
13     }
14 }
```



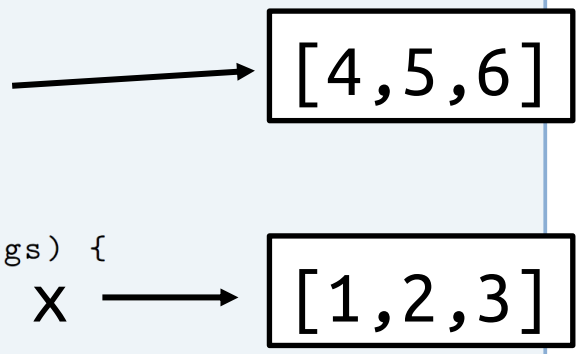
Value vs Reference - Beispiel

```
1  import java.util.Arrays;
2
3  public class Example {
4      public static void setX(int[] x) {
5          x = new int[] {4, 5, 6};
6      }
7
8      public static void main(String[] args) {
9          int[] x = new int[] {1, 2, 3};
10         setX(x);
11         String xStr = Arrays.toString(x);
12         System.out.println(xStr);
13     }
14 }
```



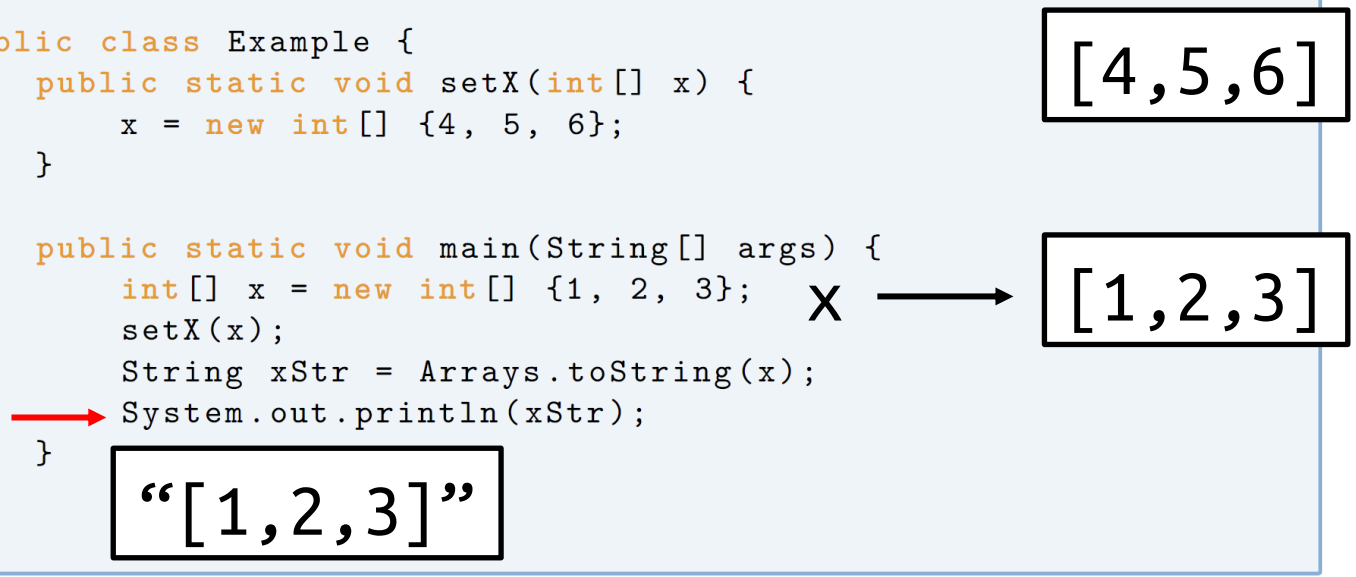
Value vs Reference - Beispiel

```
1  import java.util.Arrays;
2
3  public class Example {
4      public static void setX(int[] x) {
5          x = new int[] {4, 5, 6};
6      }
7
8      public static void main(String[] args) {
9          int[] x = new int[] {1, 2, 3};
10         setX(x);
11         String xStr = Arrays.toString(x);
12         System.out.println(xStr);
13     }
14 }
```



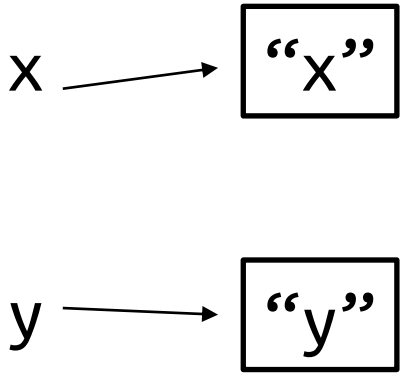
Value vs Reference - Beispiel

```
1  import java.util.Arrays;
2
3  public class Example {
4      public static void setX(int[] x) {
5          x = new int[] {4, 5, 6};
6      }
7
8      public static void main(String[] args) {
9          int[] x = new int[] {1, 2, 3};
10         setX(x);
11         String xStr = Arrays.toString(x);
12         System.out.println(xStr);
13     }
14 }
```



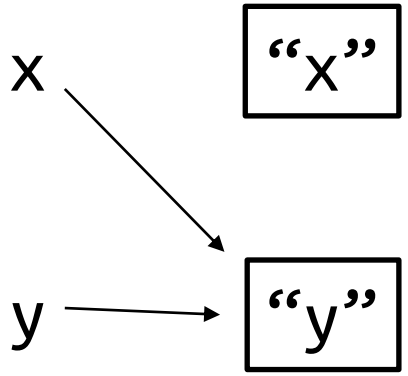
Value vs Reference - Beispiel

```
1  String x = "x";
2  String y = "y";
3
4  //Assignment
5  x = y;
```



Value vs Reference - Beispiel

```
1  String x = "x";
2  String y = "y";
3
4  //Assignment
5  x = y;
```



Quiz: Referenzen

```
int[] a = {1, 2};
int[] b = a;
a = null;
System.out.println(b[0]);
```

Rekursion

Rekursion in Java – Aufgabe

Erstelle eine Klasse `CheckIsPalindromRecursive`.
Implementiere eine **statische, rekursive Methode** `IsPalindrome`, die:

- Einen String akzeptiert
- Einen **boolean** zurückgibt, ob der gesamte String ein Palindrom ist.

Basisfall der Rekursion:

- Wenn die String-Länge 0 oder 1 beträgt, gib `true` zurück
- Bei einer Länge von 2: Prüfe, ob beide Zeichen gleich sind, und gib entsprechend `true` oder `false` zurück.

Rekursiver Fall:

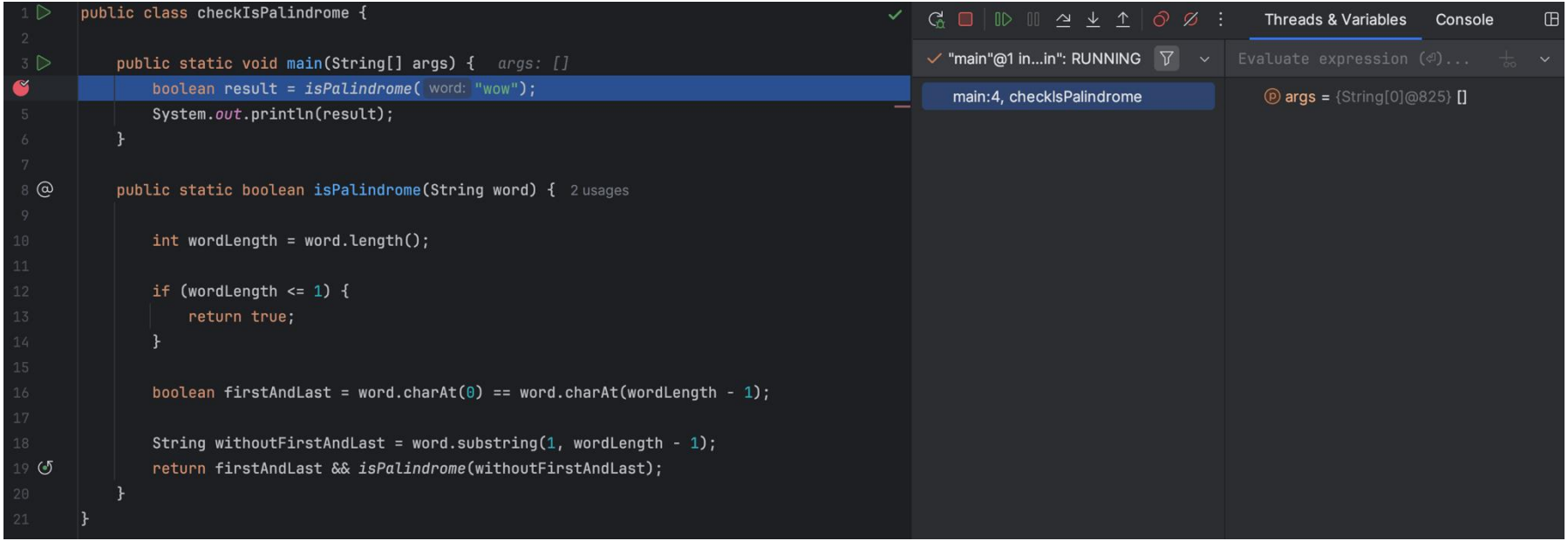
- Nimm das **erste** und **letzte** Zeichen des Strings.
- Überprüfe, ob sie **gleich** sind.
- Verknüpfe das Ergebnis logisch mit einem Aufruf von `IsPalindrome` für den Teilstring ohne das erste und letzte Zeichen.

Tipp: Benutze `str.charAt(i)`, `str.substring(i, j)`

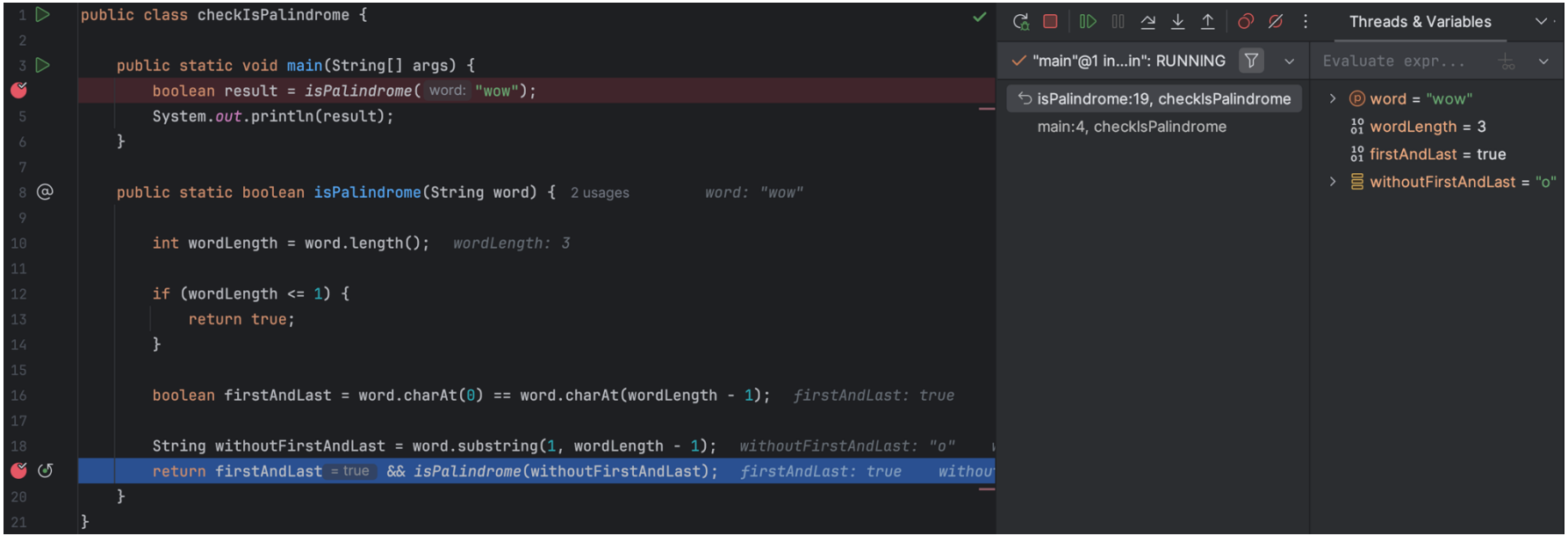
Rekursion in Java – Lösung

```java
public class Palindrome {
    public static boolean isPalindrome(String word) {
        if (word.length() <= 1)
            return true;
        boolean firstAndLast = word.charAt(0) == word.charAt(word.length() - 1);
        String withoutFirstAndLast = word.substring(1, word.length() - 1);
        return firstAndLast && isPalindrome(withoutFirstAndLast);
    }
}
```

Rekursion mit Debugger

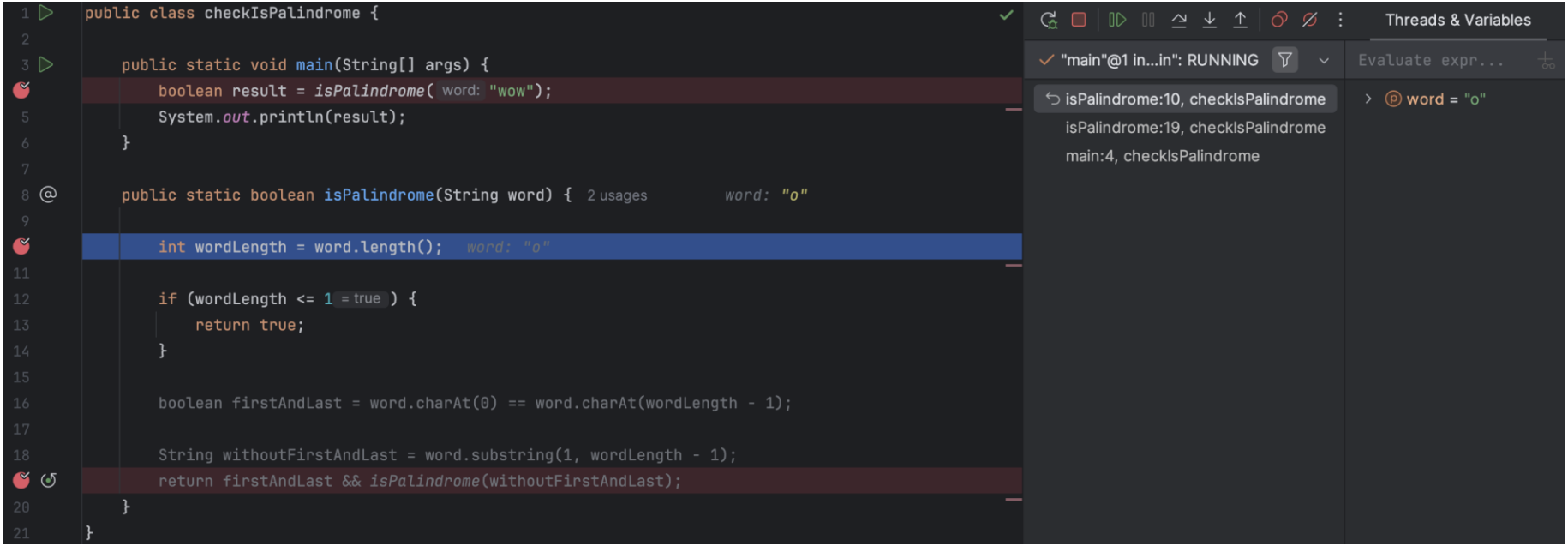


- Wir setzen den Breakpoint dort, wo die Methode **`isPalindrome`** aufgerufen wird. Anschliessend gehen wir mit 'Step Into' in die Methode, wobei das Argument **`word = "wow"`** ist.



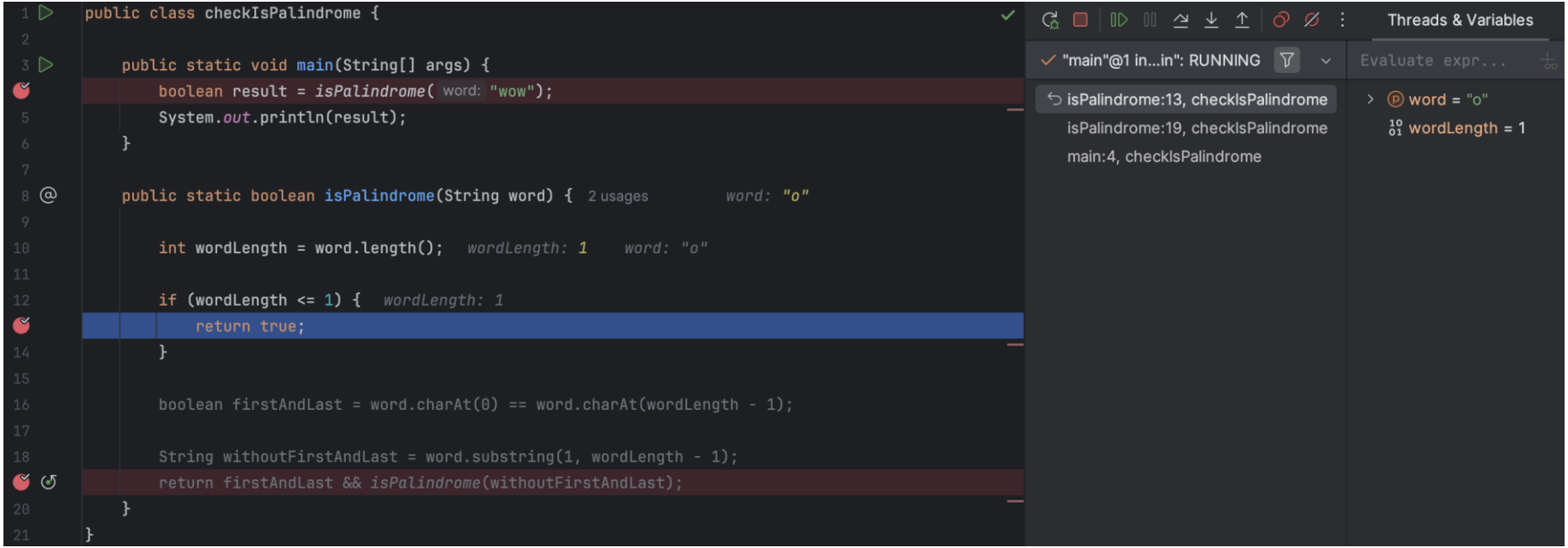
- Wir gehen mit 'Step Over' durch die Methode, und die Variablen **word**, **wordLength**, **firstAndLast** erscheinen sowohl unter „Threads & Variables“ als auch in der Zeile , nachdem sie deklariert wurden.

- In Zeile 19 rufen wir die Methode **isPalindrome** erneut auf, diesmal innerhalb der Methode **isPalindrome** (wir gehen in die Rekursion). Das Argument in diesem Aufruf der Methode ist **withoutFirstAndLast** (="o").



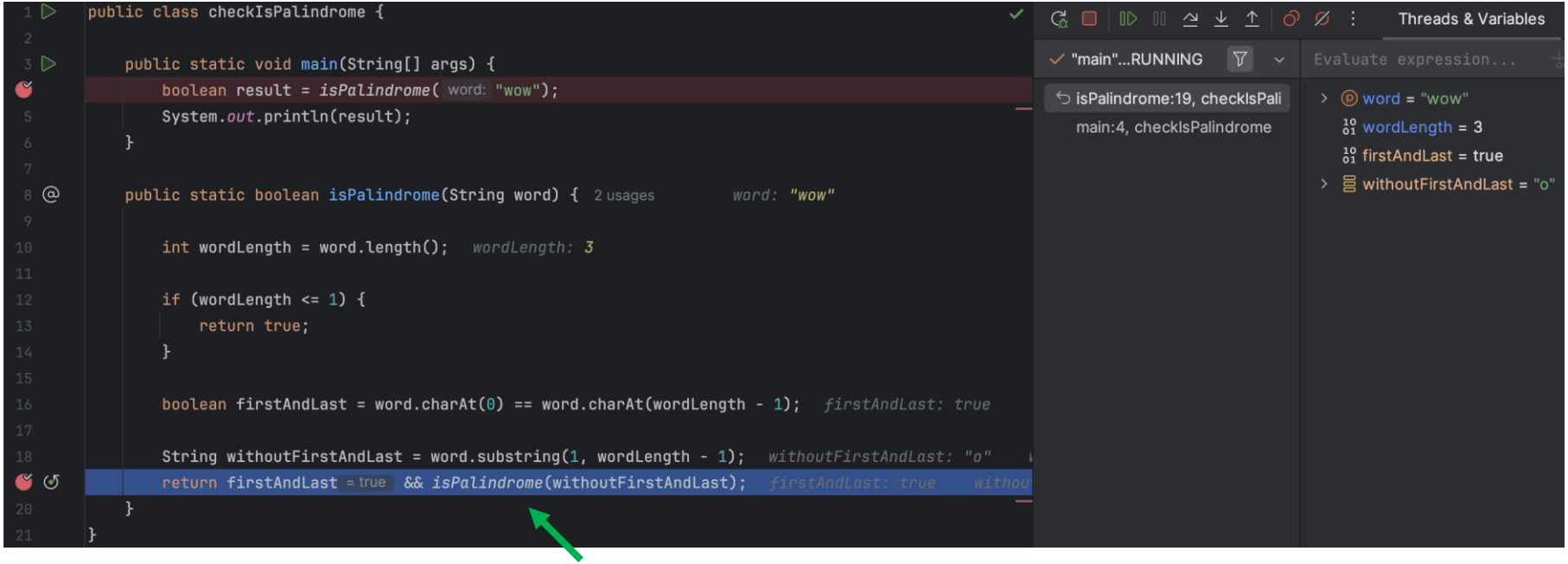
- Wir sind wieder in die Methode eingetreten, aber dieses Mal mit **`word`** = "o".

- Beachte, dass die Werte, die im vorherigen Methodenaufruf von isPalindrome sichtbar waren, nicht mehr im Scope sind, und daher auch nicht in unter „Threads & Variables“ .



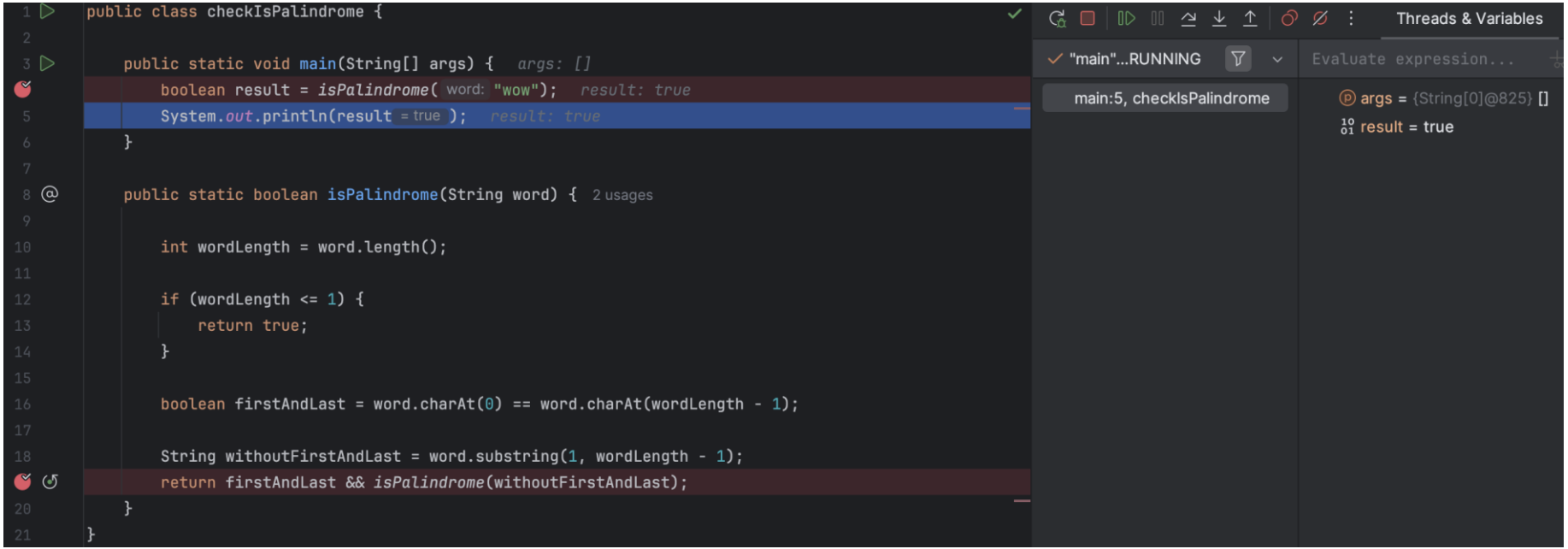
- "Da **`wordLength = 1`** ist, ist der Base Case erfüllt. Somit betreten wir das **if-Statement** und es wird **`true`** zurückgegeben.

- Aber wohin wird es zurückgegeben?

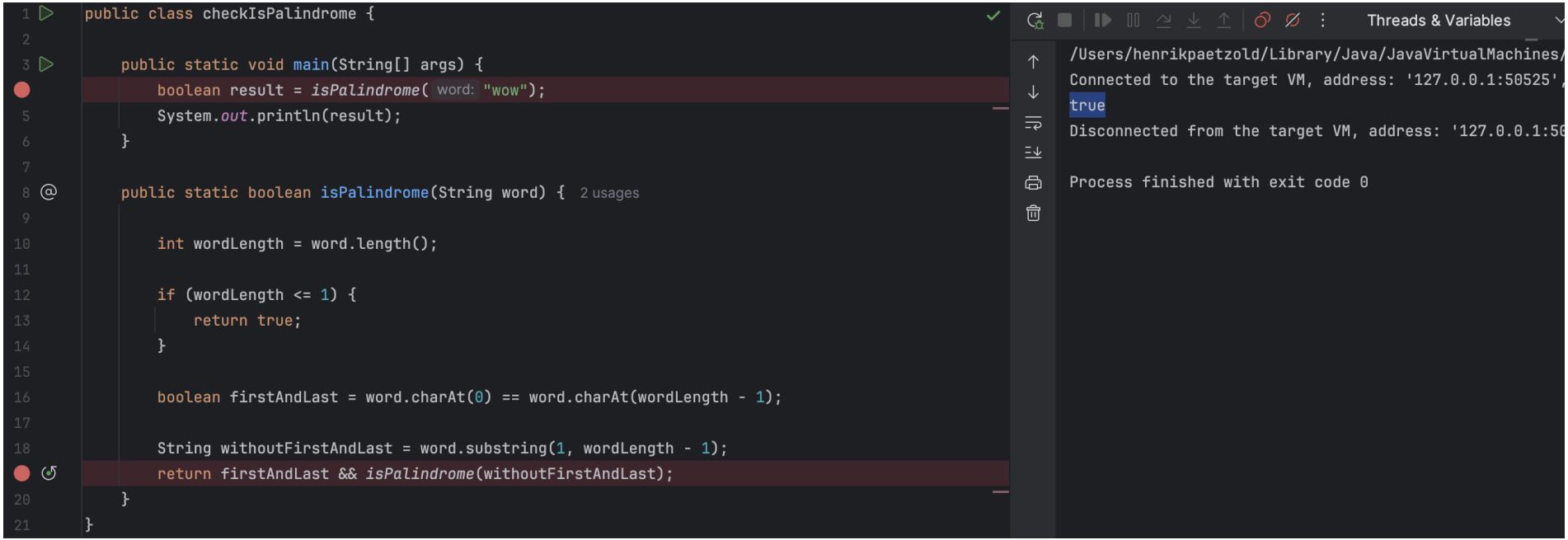


- **True** wird an die Stelle zurückgegeben, an der die Methode zuletzt aufgerufen wurde. Die Methode wurde zuletzt mit **'o'** aufgerufen. Nun zeigt auch „Threads & Variables“ wieder die Werte der Variablen, wie sie beim vorherigen Methodenaufruf waren.

- Jetzt wird zuerst **firstAndLast && True** (Rückgabewert) ausgewertet. Das ergibt **True**, was weiter zurückgegeben wird, an die Stelle, an der isPalindrome mit **word = "wow"** aufgerufen wurde.



- **`isPalindrome`** wurde mit dem Parameter **`"wow"`** in der **`main-Methode`** aufgerufen. **`True`** wird also hier zurückgegeben, und anschliessend wird **`System.out.println(true)`** ausgeführt.


public class checkIsPalindrome {
public static void main(String[] args) {
boolean result = isPalindrome(word: "wow");
System.out.println(result);
}
public static boolean isPalindrome(String word) { 2 usages
int wordLength = word.length();
if (wordLength <= 1) {
return true;
}
boolean firstAndLast = word.charAt(0) == word.charAt(wordLength - 1);
String withoutFirstAndLast = word.substring(1, wordLength - 1);
return firstAndLast && isPalindrome(withoutFirstAndLast);
}
}
Threads & Variables
/Users/henrikpaetzold/Library/Java/JavaVirtualMachines/
Connected to the target VM, address: '127.0.0.1:50525',
true
Disconnected from the target VM, address: '127.0.0.1:50
Process finished with exit code 0

Was wird hier passieren?

```java
public class checkIsPalindrome {
    public static void main(String[] args) {
        boolean result = isPalindrome("baab");
        System.out.println(result);
    }

    public static boolean isPalindrome(String word) {
        int wordLength = word.length();
        boolean firstAndLast = word.charAt(0) == word.charAt(wordLength - 1);
        String withoutFirstAndLast = word.substring(1, wordLength - 1);
        return isPalindrome(withoutFirstAndLast) && firstAndLast;
    }
}
```

Error? Endlose Rekursion? Korrektes ausführen? Sonstiges?

Was wird hier passieren?

```java
public class checkIsPalindrome {
    public static void main(String[] args) {
        boolean result = isPalindrome("wow");
        System.out.println(result);
    }

    public static boolean isPalindrome(String word) {
        int wordLength = word.length();
        boolean firstAndLast = false;
        String withoutFirstAndLast = "";
        if (wordLength != 1 && wordLength != 0) {
            withoutFirstAndLast = word.substring(1, wordLength - 1);
            firstAndLast = word.charAt(0) == word.charAt(wordLength - 1);
        }
        return isPalindrome(withoutFirstAndLast) && firstAndLast;
    }
}
```

Was wird hier passieren?

```java
public class checkIsPalindrome {
    public static void main(String[] args) {
        boolean result = isPalindrome("wow");
        System.out.println(result);
    }

    public static boolean isPalindrome(String word) {
        int wordLength = word.length();
        if (wordLength < 1) {
            return true;
        }
        boolean firstAndLast = false;
        String withoutFirstAndLast = "";
        if (wordLength != 1 && wordLength != 0) {
            withoutFirstAndLast = word.substring(1, wordLength - 1);
            firstAndLast = word.charAt(0) == word.charAt(wordLength - 1);
        }
        return isPalindrome(withoutFirstAndLast) && firstAndLast;
    }
}
```

Arrays

Prüfungsaufgabe HS24

Geben Sie den Wert von `sum` **nach jeder Iteration** der for-Schleife an. Was ist die Ausgabe der Methode `main` der Klasse `Main`?

```
class Main {
  public static void main(String[] args) {

    int[] a = new int[]{10, 20, 30, 40, 50};

    int sum = 0;

    for (int i = 0; i < a.length; i++) {
      if (i % 2 == 0) {
        sum += a[i];
      } else {
        sum -= i++;
      }
    }
    System.out.println("sum = " + sum);
  }
}
```

Werte von `sum` nach jeder Iteration:

_____

_____

_____

_____

_____

Ausgabe:

`sum =` _____

Prüfungsaufgabe HS24

Geben Sie den Wert von sum **nach jeder Iteration** der for-Schleife an. Was ist die Ausgabe der Methode main der Klasse Main?

```
class Main {
  public static void main(String[] args) {

    int[] a = new int[]{10, 20, 30, 40, 50};

    int sum = 0;

    for (int i = 0; i < a.length; i++) {
      if (i % 2 == 0) {
        sum += a[i];
      } else {
        sum -= i++;
      }
    }
    System.out.println("sum = " + sum);
  }
}
```

Werte von sum nach jeder Iteration:

Iteration 1: sum = 10 (i = 1)

Iteration 2: sum = 9 (i = 3)

Iteration 3: sum = 6 (i = 5)

Ausgabe:

sum = 6

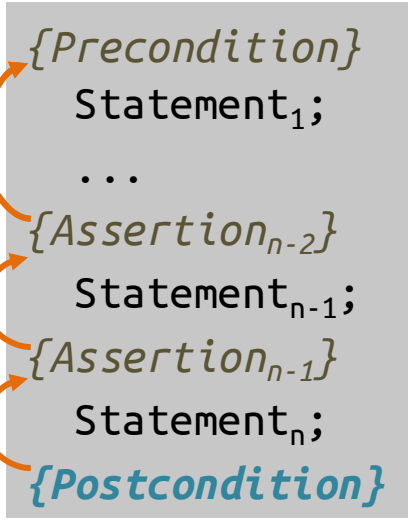
Logisches Schliessen I

Stärkere und schwächere Aussagen

- Wir können *stärkere* und *schwächere* Aussagen unterscheiden
  - Wenn $P_1 \Longrightarrow P_2$ gilt, dann ist P_1 stärker als P_2 (und P_2 schwächer als P_1)

- Die stärkste Aussage ist `false`, da `false` alles impliziert
- Die schwächste Aussage ist `true`, da `true` nur `true` impliziert

Rückwärtsschliessen: Vorgehen



- **Start:** Wählen (wissen, raten) einer sinnvollen *Nachbedingung*
- **Schrittweise:** Herleiten der vorherigen Aussage (`Assertion`$_{i-1}$) durch Einbeziehen des *Effekts* der vorherigen Anweisung (`Statement`$_i$)
- **Ziel:** Herleiten einer *notwendigen und hinreichenden Vorbedingung*

- Rückwärts = «Welche Vorbedingung braucht mein Code, damit er die gewählten Garantien (Nachbedingung) geben kann?»

Weakest Precondition

- Die **schwächste Vorbedingung (weakest precondition)** ist die schwächste Vorbedingung, die die Postcondition impliziert.
  - Falls die Postcondition `{ true }` ist, so ist `{ true }` die schwächste Vorbedingung. Alles impliziert die Postcondition, also insbesondere auch die schwächste Bedingung `true`.
  - Falls die Postcondition `{ false }` ist, so ist `{ false }` die schwächste Vorbedingung. Nur `{ false }` impliziert die Postcondition, demensprechend ist es die schwächste (und einzige) Vorbedingung.
- Die vorgestellten Regeln fürs Rückwärtsschliessen ergeben direkt die schwächsten Vorbedingungen.

Weakest Precondition – Einfaches Beispiel

{ }

$x = y*y$ ———

{x>4}

Weakest Precondition – Einfaches Beispiel

{ }

$x = y*y$ ——— $\{x>4\}$

Weakest Precondition – Einfaches Beispiel

{ }

x = y*y ——————— {y*y>4}

Weakest Precondition – Einfaches Beispiel

{y*y>4}

x = y*y ————

Weakest Precondition – Einfaches Beispiel

$\{|y|>2\}$

$x = y*y$ ————

Weakest Precondition – Zweites Beispiel

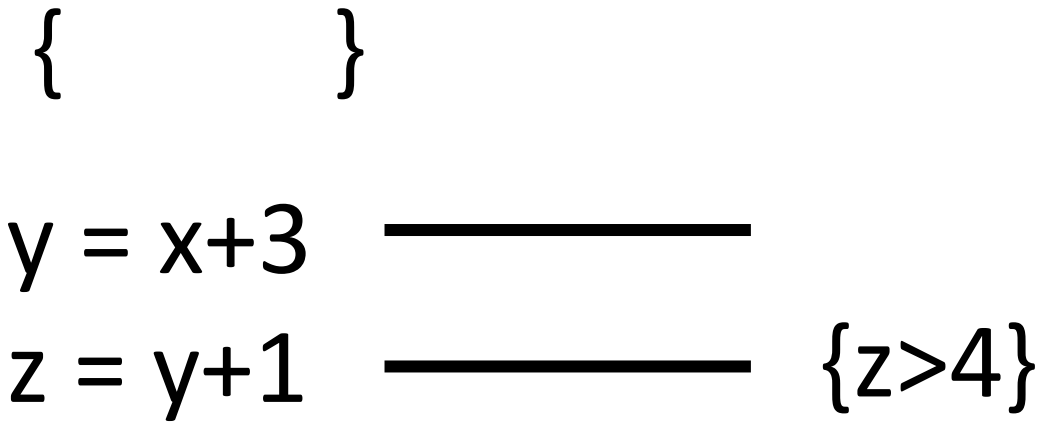
{ }

y = x+3 ———

z = y+1 ———

{z>4}

Weakest Precondition – Zweites Beispiel



Weakest Precondition – Zweites Beispiel

{ }

y = x+3 ——————

z = y+1 ——————— {y+1 >4}

Weakest Precondition – Zweites Beispiel

{ }

y = x+3 ——— {y+1 >4}

z = y+1 ———

Weakest Precondition – Zweites Beispiel

{ }

$y = x+3$ ——— $\{x+3+1 > 4\}$

$z = y+1$ ———

Weakest Precondition – Zweites Beispiel

{ }

$y = x+3$ ——— {x >0}

$z = y+1$ ———

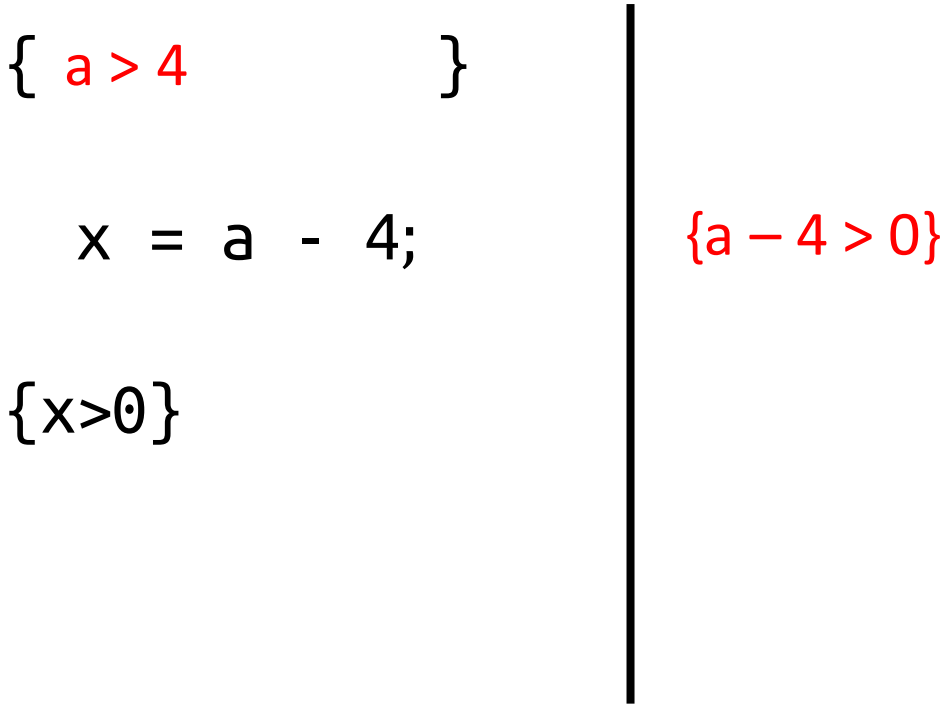
Weakest Precondition – Zweites Beispiel

$\{x > 0\}$

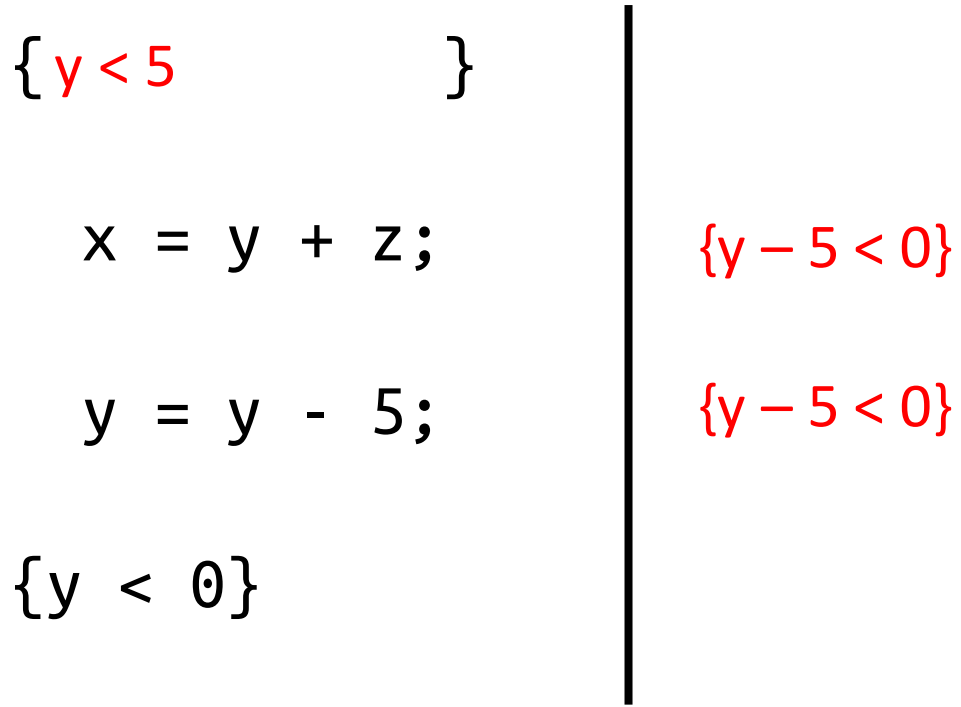
$y = x+3$ ——————

$z = y+1$ ——————

Weakest Precondition - Beispiel



Weakest Precondition - Beispiel



Weakest Precondition– Prüfungsbeispiel

```
{ b > 10 && a > -b    }

  w = a

  x = w + b;

  y = x * 2;

{y > 0 && b > 10}
```

{(a + b) * 2 > 0 && b > 10}

{(w + b) * 2 > 0 && b > 10}

{x * 2 > 0 && b > 10}

Weakest Precondition– Prüfungsbeispiel

```
{ q > 14 / 3                }

  p = 3 * q

  p = p + 1;

{p > 15}
```

Alternative: {q >= 5}

{ 3 * q + 1 > 15}

{ p + 1 > 15}

Nachbesprechung

Aufgabe 1: Sieb des Eratosthenes

Schreiben Sie ein Programm "Sieb.java", das eine Zahl *limit* einliest und die Anzahl der Primzahlen, die grösser als 1 und kleiner oder gleich dem *limit* sind, ausgibt. Dazu ermitteln Sie in einem ersten Schritt alle Primzahlen, die kleiner oder gleich *limit* sind. Dieses Teilproblem können Sie mit dem Sieb des Eratosthenes lösen. Das Sieb des Eratosthenes findet Primzahlen bis n. Man betrachtet alle Zahlen von 2 bis n und streicht zuerst alle Vielfachen der ersten Zahl (2). Dann geht man zur nächsten ungestrichenen Zahl (3) und wiederholt das Streichen ihrer Vielfachen. Das macht man, bis man dies für alle Zahlen gemacht hat. Sie können ein `Boolean`-Array verwenden, um zu speichern, welche Zahlen Primzahlen sind und welche nicht. Übrig bleiben die Primzahlen. Danach können Sie die Anzahl der gefundenen Primzahlen anhand dieses Arrays bestimmen.

Beispiel: Für $limit = 13$ sollte Ihr Programm 6 ausgeben (Primzahlen: 2, 3, 5, 7, 11, 13).

Hinweis: Es ist nicht zwingend nötig von 2 bis n zu gehen. Von 2 bis $\sqrt{n}$ zu gehen reicht bereits aus, da eine Zahl $\leq n$ nicht einen Teiler grösser als $\sqrt{n}$ ausser sich selbst haben kann.

Aufgabe 2: Arrays

1. Implementieren Sie die Methode `ArrayUtil.zeroInsert(int[] x)` in der Datei "ArrayUtil.java". Die Methode nimmt einen Array `x` als Argument und gibt einen Array zurück. Der zurückgegebene Array soll die gleichen Werte wie `x` haben, ausser: Wenn eine positive Zahl direkt auf eine negative Zahl folgt oder wenn eine negative Zahl direkt auf eine positive Zahl folgt, dann wird dazwischen eine 0 eingefügt.

   Beispiele:

   - Wenn `x` gleich `[3, 4, 5]` ist, dann wird `[3, 4, 5]` zurückgegeben.
   - Wenn `x` gleich `[3, 0, -5]` ist, dann wird `[3, 0, -5]` zurückgegeben.
   - Wenn `x` gleich `[-3, 4, 6, 9, -8]` ist, dann wird `[-3, 0, 4, 6, 9, 0, -8]` zurückgegeben.

   Versuchen Sie, die Methode rekursiv zu implementieren.

Aufgabe 2: Arrays

2. Implementieren Sie die Methode `ArrayUtil.tenFollows(int[] x, int index)`. Die Methode gibt einen Boolean zurück. Die Methode soll `true` zurückgeben, wenn im Array `x` ab Index `index` der zehnfache Wert einer Zahl `n` direkt der Zahl `n` folgt. Dies muss nur für das erste Auftreten der Zahl `n` ab Index `index` im Array `x` geprüft werden. Ansonsten soll die Methode `false` zurückgeben.

   Beispiele:

   - `tenFollows([1, 2, 20], 0)` gibt `true` zurück.
   - `tenFollows([1, 2, 7, 20], 0)` gibt `false` zurück.
   - `tenFollows([3, 30], 0)` gibt `true` zurück.
   - `tenFollows([3], 0)` gibt `false` zurück.
   - `tenFollows([1, 2, 20, 5], 1)` gibt `true` zurück.
   - `tenFollows([1, 2, 20, 5], 2)` gibt `false` zurück.

   Die `main` Methode in `ArrayUtil` gibt die oben genannten Beispielaufrufe sowie das entsprechende Ergebnis der jeweiligen Methode aus. Hiermit können Sie überprüfen, ob Ihre Implementierungen die richtigen Ergebnisse zurückliefern. In "ArrayUtilTest.java" im Ordner "test" in der Übungsvorlage finden Sie zusätzlich einige Unit-Tests für beide Methoden (für eine detaillierte Beschreibung zu automatisiertem Testen und der Ausführung solcher Tests siehe Aufgabe 3). Sie können die `main` Methode und die Tests beliebig abändern und/oder mit Ihren eigenen Inputs erweitern.

Aufgabe 3: 2D Arrays

Gegeben einer Matrix M, prüfen Sie zuerst ob diese eine $n \times n$ Matrix ist, deren Elemente positive ganze Zahlen sind. Danach prüfen Sie ob zusätzlich alle Zahlen kleiner gleich n^2 sind. Somit gilt nun $0 < m_{i,j} < n^2$. Prüfen Sie ebenfalls, ob die Elemente der Matrix jeweils genau einmal vorkommen, sprich ob $m_{x,y} = m_{p,q} \Rightarrow (x = p) \wedge (y = q)$ gilt. Wir sagen, dass die Matrix M *perfekt* ist, wenn zusätzlich alle Zeilensummen und Spaltensummen gleich sind (also $\sum_{k=0}^{k=n-1} m_{i,k} = \sum_{k=0}^{k=n-1} m_{j,k}$ für alle i, j und $\sum_{k=0}^{k=n-1} m_{k,i} = \sum_{k=0}^{k=n-1} m_{k,j}$ für alle i, j mit $0 \leq i, j < n$).

Vervollständigen Sie die Methode `boolean checkMatrix(int[][] m)` von der Klasse Matrix, so dass diese Methode `true` zurückgibt wenn die Input Matrix *perfekt* ist, und `false` sonst. Sie können davon ausgehen, dass der Parameter `m` nicht `null` ist. Alle anderen Eigenschaften müssen Sie selber testen. Eine Matrix ist nur perfekt, wenn alle genannten Eigentschaften gelten.

Testen Sie Ihr Programm ausgiebig - am besten mit JUnit - und pushen Sie die Lösung vor dem Abgabetermin. Wir haben Ihnen einen JUnit Test in der Klases `MatrixTest` bereits erstellt.

Aufgabe 4: Testen mit JUnit

- **Zweck des Programms:**
  - Wochentag eines Datums (nach 01.01.1900) ausgeben
    - Beispiel: 13.10.2017 -> Friday
    - Gibt fälschlicherweise aber “The 13.10.2017 is a Sunday” aus.
  - Berücksichtigt Schaltjahre (“Leap Year”)
- **Funktionsweise:**
  1. Überprüft, ob das Datum OK ist
  2. Zählt die Tage ab 01.01.1900 bis zum eingegebenen Datum
  3. Wochentag = Tage % 7

Aufgabe 5: Matching Numbers

Implementieren Sie die Methode `Match.matchNumber(long A, int M)`. Die Methode soll für eine Zahl `A` und eine nicht-negative drei-stellige Zahl `M` die Position von `M` in `A` zurückgeben. Sei `M` eine Zahl mit den Ziffern $M_2M_1M_0$ (das heisst, es gilt $M = M_0 + 10 \cdot M_1 + 100 \cdot M_2$), wobei jede Ziffer 0 sein kann. Zusätzlich sei `A` eine Zahl, sodass A_i die i-te Ziffer von `A` ist (das heisst, es gilt $|A| = \sum_i 10^i \cdot A_i$), wobei `A` unendlich viele führende Nullen hat. Die Position von `M` in `A` ist die kleinste Zahl j, sodass $A_j = M_0$ und $A_{j+1} = M_1$ und $A_{j+2} = M_2$ gilt. Die Methode soll `-1` zurückgeben, falls es kein solches j gibt.

Beispiele:

`matchNumber(32857890, 789)` soll `1` zurückgeben.
`matchNumber(37897890, 789)` soll `1` zurückgeben.
`matchNumber(1800765, 7)` soll `2` zurückgeben.
`matchNumber(1800765, 8)` soll `-1` zurückgeben (die drei Ziffern von `8` sind `008`).
`matchNumber(75, 7)` soll `1` zurückgeben (da `007` and Position `1` von `0075` ist).

Aufgabe 5: Matching Numbers

Implementieren Sie die Berechnung in der Methode `int matchNumber(long A, int M)`, welche sich in der Klasse `Match` befindet. Die Deklaration der Methode ist bereits vorgegeben. Sie können davon ausgehen, dass $0 \leq \texttt{M} < 1000$ gilt.

In der `main` Methode der Klasse `Match` finden Sie die oberen Beispiele als kleine Tests, welche Beispiel-Aufrufe zur `matchNumber`-Methode machen und welche Sie als Grundlage für weitere Tests verwenden können. In der Datei `MatchTest.java` geben wir die gleichen Tests zusätzlich auch als JUnit Test zur Verfügung. Sie können diese ebenfalls nach belieben ändern. Es wird *nicht* erwartet, dass Sie für diese Aufgabe den JUnit Test verwenden.

Tipp: Die Methode `Integer.toString(int i)` wandelt einen Integer in einen String um.

Vorbesprechung

Ranking

- **Prüfungsrelevant**

- **Empfehlenswert**

- **Nicht prüfungsrelevant**

ArrayUtil.zeroInsert(int[] x)

Aufgabe 1: Präfixkonstruktion

Gegeben seien zwei Strings `s` und `t` und ein Integer `n` mit $n \geq 0$. Schreiben Sie ein Programm, das zurückgibt, ob `s` eine Konkatenation von maximal `n` vielen Präfixen von `t` ist.

Beispiele:

- `s = "abcababc"`, `t = "abc"`, `n = 4`: Das Programm sollte `true` zurückgeben, da `"abc"` und `"ab"` Präfixe von `t` sind und `s` eine Konkatenation von `"abc"`, `"ab"`, `"abc"` ist.
- `s = "abcbcabc"`, `t = "abc"`, `n = 4`: Das Programm sollte `false` zurückgeben, da `"bc"` kein Präfix von `t` ist.
- `s = "abab"`, `t = "abac"`, `n = 2`: Das Programm sollte `true` zurückgeben, da `"ab"` ein Präfix von `t` ist und `s` eine Konkatenation von `"ab"`, `"ab"` ist.

Implementieren Sie die Methode `isPrefixConstruction(String s, String t, int n)` in der Klasse `PrefixConstruction`. Die Methode hat drei Argumente: die beiden Strings `s` und `t` und der Integer `n`. Sie dürfen davon ausgehen, dass der Integer grösser oder gleich 0 ist. In der Datei "PrefixConstructionTest.java" finden Sie Tests.

Tipp: Lösen Sie die Aufgabe rekursiv.

Tipp: Prüfe s = "ababa" t = "aba" n = 2 auf Papier.

Aufgabe 5.2: Arrays

Falls Aufgabe 6.1 zu schwierig: Löst zuerst diese Aufgabe!

1. Implementieren Sie die Methode `ArrayUtil.zeroInsert(int[] x)` in der Datei "ArrayUtil.java". Die Methode nimmt einen Array `x` als Argument und gibt einen Array zurück. Der zurückgegebene Array soll die gleichen Werte wie `x` haben, ausser: Wenn eine positive Zahl direkt auf eine negative Zahl folgt oder wenn eine negative Zahl direkt auf eine positive Zahl folgt, dann wird dazwischen eine 0 eingefügt.

   Beispiele:

   - Wenn `x` gleich `[3, 4, 5]` ist, dann wird `[3, 4, 5]` zurückgegeben.
   - Wenn `x` gleich `[3, 0, -5]` ist, dann wird `[3, 0, -5]` zurückgegeben.
   - Wenn `x` gleich `[-3, 4, 6, 9, -8]` ist, dann wird `[-3, 0, 4, 6, 9, 0, -8]` zurückgegeben.

   Versuchen Sie, die Methode rekursiv zu implementieren.

Aufgabe 2: Weakest Precondition

Bitte geben Sie für die folgenden Programmsegmente die schwächste Vorbedingung (weakest precondition) an. Bitte verwenden Sie Java-Syntax. Alle Anweisungen sind Teil einer Java Methode. Alle Variablen sind vom Typ `int` und es gibt keinen Overflow.

1.
```
P: { ?? }
S:   m = n * 4;  k = m - 2;
Q: { n > 0 && k > 5 }
```

2.
```
P: { ?? }
S:   m = n * n;  k = m * 2;
Q: { k > 0 }
```

3.
```
P: { ?? }
S:   y = x + 3; z = y + 1;
Q: { z > 4 }
```

4.
```
P: { ?? }
S:   y = x + 1; z = y - 3;
Q: { z == 10 }
```

Beispiel

```
P: { ?? }
S:   a = b * 3; c = a + 1;
Q: { a > 0 && c < 5 }
```

Aufgabe 3: Wörter Raten

Das Programm "WoerterRaten.java" enthält Fragmente eines Rate-Spiels, welches Sie vervollständigen sollen. In dem Spiel wählt der Computer zufällig ein Wort w aus einer Liste aus und der Mensch muss versuchen, das Wort zu erraten. In jeder Runde kann der Mensch eine Zeichenfolge z (welche einen oder mehrere Buchstaben enthält) eingeben und der Computer gibt einen Hinweis dazu. Folgende Hinweise sind möglich:

1. w beginnt mit z
2. w endet mit z
3. w enthält z
4. w enthält nicht z

```
Tipp? e
Das Wort enthält nicht "e"!
Tipp? a
Das Wort endet mit "a"!
Tipp? j
Das Wort beginnt mit "j"!
Tipp? v
Das Wort enthält "v"!
Tipp? java
Das Wort ist "java"!
Glückwunsch, du hast nur 5 Versuche benötigt!
```

Aus Textdatei lesen

woerter.txt

```
8
wort
blasinstrument
computer
schlange
java
programmieren
welt
sugus
```

Code zum Einlesen der Wörter:

```
Scanner scanner = new Scanner(new File("woerter.txt"));
String[] woerter = new String[scanner.nextInt()];
for(int i = 0; i < woerter.length; i++) {
    woerter[i] = scanner.next();
}
```

Anzahl Wörter = 8

Liest das nächste Wort

Absoluter Pfad

- beginnt mit `C:\...` unter Windows oder `/...` unter Linux/macOS

Relativer Pfad

- ist relativ zum aktuellen Verzeichnis (“working directory”) des Programms (bei uns der Projektordner)

Aufgabe 4: Datenanalyse

In dieser Aufgabe werden Sie die Kelchblattlänge von Iris Blumen, welche auch Schwertlilien genannt werden, analysieren. Dazu verwenden Sie ein öffentlich zugängliches Dataset [1] welches die Längen vom Kelchblatt (Sepal Length) von jeweils 150 verschiedenen Iris Blumen enthält. Es gibt sehr viele Arten von Iris Blumen, insgesamt sind 285 Arten bekannt. Diese zu unterscheiden ist ein komplexer Prozess. Wir werden jedoch versuchen mittels der Länge des Kelchblattes drei Arten von Iris Blumen zu unterscheiden, in dem wir die gegebenen Daten analysieren. Wir werden uns auf die folgenden drei Iris Blumen konzentrieren:

- Iris setosa, auch Borsten-Schwertlilie genannt.
- Iris versicolor, auch Verschiedenfarbige Schwertlilie gennannt.
- Iris virginica, auch blaue Sumpfschwertlilie genannt.



Iris Versicolor **Iris Setosa** **Iris Virginica**

Aufgabe 4: Datenanalyse

1. Das Programm soll als erstes die Kelchblattgrössen für die drei Arten von Iris Blumen aus den Dateien "sepal_length_setosa.txt", "sepal_length_versicolor.txt", und "sepal_length_virginica.txt" im Projekt-Verzeichnis in ein Array einlesen. Die Dateien haben ein ähnliches Format wie die "woerter.txt"-Datei der letzten Aufgabe. Die Kelchblattgrössen liegen als ganze Zahlen in Millimetern [mm] vor. Die erste Zahl gibt die Anzahl Daten im File an. Implementieren Sie dazu die Methode `liesLaengen()`, indem Sie die nötigen Daten aus dem gegebenen `Scanner` auslesen. Falls Sie Schwierigkeiten dabei haben, können Sie sich an der `WoerterRaten.liesWoerter()`-Methode orientieren. Sie können Ihre Methode anhand eines der drei Files testen um zu schauen, ob diese funktioniert.

   Verwenden Sie den Test *testLiesLaengen* in der Datei "DatenAnalyseTest.java", um Ihren Code zu testen.

Aufgabe 4: Datenanalyse

2. Führen Sie als nächstes eine einfache Analyse der Daten durch, indem Sie das Minimum, das Maximum, den Durchschnitt und ausserdem die Anzahl der Kelchblattgrössen ausgeben. Füllen Sie dazu die Methode `einfacheAnalyse()` aus. Beachten Sie folgende Methoden: `Math.min()` und `Math.max()`.

Beispiel:

```
Anzahl Daten: 50
Minimum: 43 mm
Maximum: 58 mm
Durchschnitt: 50 mm
```

Aufgabe 4: Datenanalyse

3. Die drei Werte, die Sie in 2. berechnet haben, sagen nicht viel über die Daten aus. Um die Daten besser zu verstehen, soll Ihr Programm ein Histogramm berechnen und auf der Konsole ausgeben. Die Textausgabe könnte ungefähr so aussehen:

```
[40,43)
[43,46)
[46,49)
[49,52) |
[52,55)
[55,58) ||
[58,61) ||||||
[61,64) ||||||||||
[64,67) |||||||||
[67,70) ||||||||||
[70,73) ||||
[73,76) ||
[76,79) |||||
[79,82) |
[82,85)
```

Implementieren Sie die Methode `histogrammAnalyse()`. Diese Methode soll zuerst den Benutzer nach der Anzahl der Histogramm-Klassen fragen, dann das Histogramm berechnen und schliesslich ausgeben. Zwei (leere) Methoden sind schon vorgegeben: `erstelleHistogramm()` und `klassenBreite()`. Für diese beiden Methoden existieren Tests in "DatenAnalyseTest.java" und Kommentare, welche Ihnen beim Schreiben des Programms helfen. Überlegen Sie sich, wie Sie das Problem weiter aufteilen möchten, und erstellen Sie entsprechende Methoden.

Bonusaufgabe

Aufgabe 5: Matrix (Bonus!)

Achtung: Diese Aufgabe gibt Bonuspunkte (siehe "Leistungskontrolle" im `www.vvz.ethz.ch`). Die Aufgabe muss eigenhändig und alleine gelöst werden. Die Abgabe erfolgt wie gewohnt per Push in Ihr Git-Repository auf dem ETH-Server. Verbindlich ist der letzte Push vor dem Abgabetermin. Auch wenn Sie vor der Deadline committen, aber nach der Deadline pushen, gilt dies als eine zu späte Abgabe. Bitte lesen Sie zusätzlich die allgemeinen Regeln.

Kahoot